CSC144V Midterm Cheatsheet

Cross-Entropy Loss for Multi-Class Classification

- The SoftMax operator $\sigma: \mathbb{R}^k \to \mathbb{R}^k$ computes a vector of predicted probabilities $\sigma(z): \mathbb{R}^k$ that sum to 1.0, from a vector of logits $z: \mathbb{R}^k$ in the last hidden layer (the penultimate layer), where k is the number of classes:
  - $\sigma(z)_i = \frac{\exp(z_i)}{\sum_{j=1}^{k} \exp(z_j)}$
- The loss function is defined as the negative log likelihood of the predicted probability corresponding to the correct label y:
  - $\text{Loss}(x, y; \theta) = -\log \sigma\big(h_\theta(x)\big)_y = -\log\left(\frac{\exp(h_\theta(x)_y)}{\sum_{j=1}^{k} \exp(h_\theta(x)_j)}\right) = \log\big(\sum_{j=1}^{k} \exp\big(h_\theta(x)_j\big)\big) - h_\theta(x)_y$
  - Minimizing $\text{Loss}(h_\theta(x), y)$ amounts to maximizing the logit $\big(h_\theta(x)\big)_y$ corresponding to the correct label y

Binary Classification Metrics

Consider a medical test that aims to diagnose people with a certain disease. "Positive" denotes sick (has disease), and "Negative" denotes healthy (no disease)
True Positive (TP): a sick person is diagnosed as sick
True Negative (TN): a healthy person is diagnosed as healthy
False Positive (FP): a healthy person is misdiagnosed as sick
False Negative (FN): a sick person is misdiagnosed as healthy

- Precision = $\frac{TP}{TP+FP}$
- Recall (True Positive Rate, TPR) = $\frac{TP}{TP+FN}$
- False Positive Rate (FPR) = $\frac{FP}{FP+TN}$
- Accuracy = $\frac{TP+TN}{TP+TN+FP+FN}$
- $F1 = 2 * \frac{\text{Precision} * \text{Recall}}{(\text{Precision}+\text{Recall})}$

Model Training vs. Local Search for Adversarial Input Generation

- To solve $\min_\theta \mathbb{E}_{(x,y)\sim D} \text{Loss}(x, y; \theta)$ for model training: gradient descent $\theta \leftarrow \theta - \alpha \nabla_\theta \text{Loss}(x, y; \theta)$
  - Update model params θ by following the gradient downhill, in order to decrease $\text{Loss}(x, y; \theta)$. ($\alpha$ is the Learning Rate)
- To solve $\max_{\delta \in \Delta} \text{Loss}(x + \delta, y; \theta)$ for adversarial input generation: gradient ascent $\delta \leftarrow \delta + \alpha \nabla_\delta \text{Loss}(x + \delta, y; \theta)$
  - Update input $x + \delta$ by following the gradient uphill, in order to increase $\text{Loss}(x + \delta, y; \theta)$, while ensuring $\delta \in \Delta$

Adversarial Robust ML

Solve $\min_{\theta} \mathbb{E}_{(x,y)\sim D} \max_{\delta\in\Delta} \text{Loss}(x+\delta, y; \theta)$

- Inner maximization problem: generating an adversarial input by adding a small perturbation $\boldsymbol{\delta}$ (or ensuring one does not exist).
- Outer minimization problem: training a robust classifier in the presence of adversarial examples. Higher network capacity enables more complex decision boundary and more robust classification